

Parallel Research on KMP Algorithm

Panwei Cao, Suping Wu
School of Mathematics and Computer Science
Ningxia University
Yinchuan Ningxia, China

Abstract—The tradition pattern matching algorithm need backtrack and compare repeatedly, so that affects efficiency of algorithm. Knuth and others put forward KMP algorithm in order to promote efficiency of the pattern matching. Paralle KMP algorithm based on MPI is provided in this paper, which can get higher efficiency.

Keywords—KMP; MPI; Parallel

I. INTRODUCTION

The string is a linear table which has a wide range of applications in computer application system, such as text editing, information retrieval, natural language translation and so on. We always need to test a specified character string whether in another string in these applications. Suppose the string s and the string t are two strings. The process of finding sub-string in string t , which the sub-string is equal to the string p , is called pattern matching. the string t is called the main string and string p is called pattern string. If we find the string p in string t , we say that the matching is successful, otherwise the matching is failed[1].

Pattern matching algorithm is applied extensively in text editing, information retrieval, spell checking, dictionary-based language translation, WWW search engines, computer virus signature matching, data compression, DNA sequence matching and other computer application system[2][3][4].

String matching contains accurate matching, fuzzy matching, parallel matching, etc. Some well-known pattern matching algorithms are BF algorithm, KMP algorithm, RK algorithm, BM algorithm and some improved algorithms[5].

II. QUESTIONS

There are two strings t (the length of the string t is n) and p (the length of the string p is m):

$$t = t_0 t_1 \dots t_{n-1}$$
$$p = p_0 p_1 \dots p_{m-1}$$

Now, our task is to find out all sub-strings which are equal to string p in string t . t is called target-string, p is called mode-string[6].

The simplest way to approach pattern matching is to compare the characters of p with the characters of t one by one. If $t_0 = p_0, t_1 = p_1, \dots, t_{m-1} = p_{m-1}$, we say matching success. Otherwise, there will exists some $i (0 \leq i \leq m-1)$, such that, $t_i \neq p_i$. Then we can right shift one character of p , compare the start character p_0 of p with character t_1 of t one by one repeatedly, until one of the following two cases occurs: if $t_i = p_0, t_{i+1} = p_1, \dots, t_{i+m-1} = p_{m-1}$, then the matching is successful; or

matching is failed.

For example, assume $t = \text{"abbaba"}$ and $p = \text{"aba"}$, the length of s is 6 ($n = 6$), the length of t is 3 ($m = 3$). The practice with the process of such pattern matching is shown in Figure 1.

t a b b a b a
|| || $\neq$
 p a b a
(a) when $p_2 \neq t_2$, right shift p one bit
 t a b b a b a
 $\neq$
 p a b a
(b) when $p_0 \neq t_1$, right shift p one bit
 t a b b a b a
 $\neq$
 p a b a
(c) when $p_0 \neq t_2$, right shift p one bit
 t a b b a b a
|| || ||
 p a b a
(d) the matching is successful

Figure 1

Simple pattern matching algorithm is easy to understand, but the efficiency is not high, if two compared characters are not equal, we will right shift one character of p and then start comparing from p_0 . This causes backtracking, this also is the reason why the efficiency of simple matching algorithm is so low. In Figure 1, we see that $p_0 \neq t_0, p_1 = t_1, p_2 \neq t_2$, and $p_0 \neq p_1$, and we have $p_0 \neq t_1$ from $p_0 \neq p_1$. So the comparative of one right shift character of p in (b) is also unequal; By the fact that $p_0 = p_2$, we have that $p_0 \neq t_2$. Hence, the comparative of one right shift character of p in (c) is also unequal. Therefore, from (a) we can be directly right shift three character to jump to (d), start the comparison from p_0 and t_3 . Matching is completed soon if we can find $p_2 \neq t_2$ in (a), we can directly jump to (d), and start the comparison from p_0 and t_3 , so that we eliminate the backtracking.

According to the above analysis we know that in matching process, once the comparison of p_i and t_j is not equal, then

$$p_0 = t_{j-1}, p_1 = t_{j-i+1}, \dots, p_{i-1} = t_{j-1}, p_i \neq t_j$$

In order to improve the speed of matching, we need to find the number of right shift character, which is not less than 1, and determine the characters of p and t to continue to compare. The most ideal situation is no backtracking in matching process of the string t . That is to say, after we right

shift some characters, we should use a new character $p_k(k < i)$ of p to compare with t_j (or even t_{j+1}).

Knuth et al. find that the value of k exists, and it depends on the model string p . That has nothing to do with the target string t . So we can calculate it before the matching of any target string.

In the following, we denote the value of k which is corresponding to p_i by $\text{next}[i]$. Its significance lies in: once the comparison of p_i and t_j is unequal in matching process, we can compare $p_{\text{next}[i]}$ with t_j . Especially, if any character of p is not necessary to compare with t_j (the sign -1 denotes $\text{next}[i]$), we directly make the comparison starting from t_{j+1} and p_0 .

For any model p , if we can determine the value of $\text{next}[i](i=0,1,\dots,m-1)$, we can accelerate the matching process as follows, when $p_i \neq t_j$, we right shift $i - \text{next}[i]$ character of p , if $\text{next}[i] \geq 0$, we start the comparison from t_j and $p_{\text{next}[i]}$, otherwise, we start the comparison from t_{j+1} and p_0 .

If we know the array of next , we have the following C language codes,

```
int pMatch(PseqString t,PseqString p,int *next)
{
    int i,j;
    i=0,j=0;
    while(i<p->n&& j<t->n)
        if(i== -1||p->[i]==t->c[j])
        {
            i++;
            j++;
        }
        else
            i=next[i];          if(i>=p->n)
                                return(j-p->n+1)
        else
            return(0);
}
```

In the study of the pattern matching algorithm without backtracking, we find that when we take the matching of characters p_i and t_j , even though is no backtracking, but we only have to take the comparison of string p and string t in order. When the string is very long, behind the characters comparing of the string t need waiting, but the front characters comparing has no irrelevant with behind the characters comparing. That means we spend a lot of time in the queue. Note that this paper is to find how many times have the string p appeared in the string t . So we should make some modifications to the above C language codes. First, we modify the loop condition of 'while' to $j < t \rightarrow n$, second, we need add a counter to count the number of matching successes. The modified code is as follow,

```
int pMatch(PseqString t,PseqString p,int *next)
{
    int i,j,count;
    i=0,j=0;
    while(j<t->n)
        if(i== -1||p->[i]==t->c[j])
        {
            i++;

```

```
            j++;
        }
        else
            i=next[i];
            if(i>=p->n)
            {
                count++;
                i=0;
            }
        return count;
}
```

III. PARALLEL ALGORITHM

Through research on the above issues, we can use parallel method to solve this problem, so that pattern matching can be faster.

The consideration of solving this problem is to create 'num' process and divide string t into 'num' parts. Then each process is distributed '(n/num)' characters. After that, pattern matching was done in every process. At last, the successful matching times of each process is counted.

Before solving specific problems, we need to declare some conditions: 1) The length of the string p should be less than or equal to the length of string t . That is: $\text{Length}(p) \leq \text{Length}(t)$; 2) The length of string which is distributed to each process should be larger than the length of p . That is, $(n/\text{num}) \geq m$.

A. Disaggregated data

As we mentioned above, we will split the string t . This is data decomposition. In the parallel algorithm, there are two ways of the decomposition of data, the cross-data decomposition and the block data decomposition. Analysis the question we have to solve, we need to choose block data decomposition in our discussion[7].

When we disaggregate data by block, we encountered two problems:

(1) After we segment the string t , we may distribute the successful matching string to the adjacent process.

Suppose that when $t = \text{cdefabghij}$, $p = \text{ab}$, $n = 10$, $m = 2$, $\text{num} = 2$, the string t is divided into two processes, the string in process 0 is $t_1 = \text{cdefa}$, its length $n_1 = 5$, the string in process 1 is $t_2 = \text{bghij}$, its length $n_2 = 5$, we will fail when we get the string p to match t_1 and t_2 . However, the string p really exists in the string t .

In order to solve this problem, we add $m-1$ to the length of all processes: (n/num) , then the length of each process is $(n/\text{num}) + m - 1$. Obviously, the last process is not need to add $m-1$. Now let's return to the above problems. The length of the string which assigned to process 0 is $n_1 = 6$, so $t_1 = \text{cdefab}$, $t_2 = \text{bghij}$, now we can do pattern matching successful in process 0.

(2) When the length of the string t is a multiple of the number of processes, then the length of string which was assigned to each process is obvious.

If n is not a multiple of num , the situation is relatively complicated. Suppose $n = 1024$, and the $\text{num} = 10$. In this case,

$1024/10 = 102.4$. If we assign 102 characters to each process, it will remain four characters. On the other hand, each process can not have 103 characters, because the string t is not so great. In this paper, we first assign $n/num=102$ characters to each process, and then assign one more character to former four process, so now the string length of four former process is 103, and the string length of remaining process is 102.

B. Implementation of parallel algorithm

Above all, we can get the corresponding parallel algorithm:

(1) We need a string t and string p . Here we read a string from a TXT document for string t , the length of string t is n . Manual input a string for string p , the length of string p is m .

(2) Calculate the next array. From no back pattern matching algorithm we know that the next array only relevant with string p , so we can calculate the next array after we input a string for p . Since the next array will be used in every process, we broadcast it to other processes by root process (we setting process 0 as the root process).

`MPI_Bcast(next,m,MPI_INT,0,comm)`

Next is the name of array, m is element number of next array.

(3) Calculate the length of each process that should be assigned, and keep them in control array, then broadcast it to all processes through `MPI_Bcast`. Each process will get the string by calculating control array.

(4) Do non-backtracking in every process, and return the number of successful matchings.

(5) The root process reduces the successful matching times of each process and prints the result.

`MPI_Reduce(&count,&Ncount,1,MPI_LONG,MPI_SUM,0,MPI_COMM_WORLD)`

Count is the successful matching times of each process, Ncount is the total number of the counts of each process.

C. Conclusion analysis

1) Experimental results

Operation platform: Intel® Core™2 Quad-cores processors. 4G memory. Software environment: Windows XP.

We search some specific string in log.txt, and the searching results and time consuming are in Table I.

TABLE I

Test text: log.txt File size: 100830352 bytes		
	Time(Second)	
Number of process	is(387760)	Microsoft(271040)
1	1.046764	1.021038
5	0.307704	0.288113
10	0.222231	0.234129
15	0.238809	0.146467
20	0.122842	0.176323
25	0.148059	0.14038
30	0.034951	0.034207
35	0.030111	0.131473
40	0.028844	0.078786
Maximum speedup	36	30

We search some specific string in log5.txt, and the searching results and time consuming are in Table II.

TABLE II

Test text: log5.txt File size: 30634631 bytes		
	Time(Second)	
Number of process	is(115263)	installation(32)
1	0.322134	0.323248
5	0.096054	0.096518
10	0.03309	0.032328
15	0.024658	0.021568
20	0.024221	0.079466
25	0.014242	0.066769
30	0.016384	0.016369
35	0.009217	0.033714
40	0.0888	0.008096
41	0.007881	0.010708
42	0.007765	0.013183
43	0.007484	0.030599
Maximum speedup	43	40

2) Results analysis

If there is only one process, we use the serial algorithm, so we can take only one process's running time as the serial algorithm running time.

With the increasing of search process number, the time for searching the target string is reduced. Actually, the speedup is not unlimited as the process increases. Under hardware conditions of this paper, we can achieve satisfactory results within 40 processes, but if more than 50 processes, communication error will occur.

IV. CONCLUSION

We combine with MPI and KMP algorithm in this paper, using MPI's Multi-process to parallel KMP algorithm. By reducing the time waiting for matching, to improve the string matching efficiency.

This algorithm implements data parallel of the KMP algorithm, next, we will combine with data parallel and algorithm parallel for better efficiency.

ACKNOWLEDGMENT

This paper is supported by the national natural science foundation of China (60963004), thanks to the national natural science foundation, thanks to the teacher's meticulously guiding, thanks to laboratory classmates' help.

REFERENCES

- [1] Na Qun Pan, Analysis on KMP Algorithm of Pattern Matching. Science Information, Vol 13, pp.335-336, 2007. (In Chinese)

- [2] Zhan Hai Yang, Research and Analysis of KMP Pattern Matching Algorithm.Comupter and Digital Engineer, Vol 38, pp.38-40, May 2010 .(In Chinese)
- [3] Wei Guo,Bai Ying Geng,Wen Jing Chen, Ameliorative Algorithm Based on KMP and Application of Warship Image Matching.Ship Electronic Engineering, Vol 6, pp.113-116, Jun 2008.(In Chinese)
- [4] Wei Qiang Nie, The Application of the KMP Algorithm in Normal Grammar.Science Mosaic, Vol 6, pp.28-30, Jan 2008.(In Chinese)
- [5] Xiao Fei Ge,Jing Wei Huang,Lei Hu, Application of Improved KMP Algorithm on the Automatic Discovery of Pattern in Biosequence.Computer Engineer, Vol 10, pp.140-142,May 2004.(In Chinese)
- [6] Nai Xiao Zhang, Algorithm and Data Struct[M].. High Education Press, 2006 .(In Chinese)
- [7] Micheal J. Quinn, Parallel Programming Of MPI and OpenMP[M]. Tsinghua University Press,2004.(In Chinese).